

Trabalho Final Inteligência Artificial I

Projeto 01: O Robô Agrícola (Agente Autônomo + Naive Bayes)

O Cenário: O aluno deverá programar um robô (um agente autônomo) que atua em uma estufa agrícola inteligente. O robô se move por um campo (uma matriz 2D) inspecionando plantas. O problema é que o robô precisa decidir o que fazer com cada planta: regar, aplicar defensivo agrícola ou ignorar.

Para tomar essa decisão, o agente conta com seus sensores (dados de umidade do solo, cor das folhas e tamanho), mas as leituras dos sensores têm incertezas.

A Abordagem Híbrida: O projeto unirá o paradigma de **Sistemas Baseados em Agentes** com **Aprendizado de Máquina Supervisionado**.

1. **O Módulo Cognitivo (Naive Bayes):** Usar o algoritmo Naive Bayes treinado com um pequeno conjunto de dados histórico (um arquivo CSV simples criado pelo próprio aluno) contendo características das plantas (Ex: *Cor da folha*, *Umidade*, *Tamanho*) e o diagnóstico real (*Saudável*, *Sede*, *Doente*). O Naive Bayes é perfeito aqui porque é estatístico, computacionalmente leve e lida muito bem com dados categóricos.
2. **O Módulo de Ação (Agente Reativo Baseado em Utilidade):** O agente terá um loop de execução clássico de IA (Percepção -> Raciocínio -> Ação). Ele deve navegar pelo campo, usar o modelo Naive Bayes para processar a percepção da planta atual, escolher a ação correta e gerenciar seus próprios recursos (ex: ele tem um tanque de água e bateria limitados).

Requisitos de Implementação (Passo a Passo)

Estruturar o código Python nas seguintes etapas:

- **Passo 1: Geração de Dados (Setup)**
 - Criar um arquivo `dataset_plantas.csv` com cerca de 100 linhas simulando dados de plantas e seus estados.
- **Passo 2: Treinamento do Modelo (Machine Learning)**
 - Usar a biblioteca `scikit-learn` (`GaussianNB` ou `CategoricalNB`) e `pandas` para carregar os dados, treinar o modelo e exportá-lo, testando sua precisão básica (acurácia).
- **Passo 3: Criação do Ambiente (Orientação a Objetos)**
 - Criar uma classe `Estufa` que represente uma matriz (ex: 5x5), onde cada célula contém uma planta gerada aleatoriamente com propriedades que os sensores do robô podem ler.
- **Passo 4: Criação do Agente Híbrido**
 - Criar uma classe `AgenteAgricultor`. Este agente vai iterar sobre a matriz da estufa. Em cada célula, ele extrai as características da planta, passa para o modelo Naive Bayes (que dirá qual é o problema da planta com base em probabilidades), e executa a ação correspondente (descontando de sua reserva de água/bateria).

Cr terios de Avalia  o

S r o avaliados os seguintes pontos:

1. **Integra  o Funcional (40%):** O agente consegue de fato utilizar o modelo treinado em tempo de execu  o para tomar decis  es?
2. **Arquitetura do C digo (30%):** O c digo est  bem modularizado? (Separa  o clara entre a l gica de aprendizado de m quina e a l gica de navega  o do agente).
3. **An lise de Resultados (30%):** O aluno consegue explicar por que o Naive Bayes classificou uma planta incorretamente em algum momento? O agente se comportou de forma racional quando a bateria ou  gua acabou?

Projeto 02 - O Robô Agrícola Evolutivo (Neuroevolução)

O Cenário: O ambiente de uma estufa (matriz 2D) com plantas precisando de cuidados. No entanto, o robô agora não recebe um histórico do que é certo ou errado. Ele possui um "cérebro" vazio e precisa aprender sozinho, através de tentativa e erro ao longo de várias gerações, como interagir com a estufa. O objetivo dele é maximizar a saúde da plantação antes que sua bateria acabe.

A Abordagem Híbrida: O projeto une a flexibilidade das **Redes Neurais Artificiais** para o processamento sensorial com o poder de busca global dos **Algoritmos Genéticos**.

1. **O Cérebro (Rede Neural Feedforward):** Uma rede neural simples recebe os estímulos dos sensores do robô em tempo real (ex: *Umidade da célula atual, Status da planta, Nível de bateria*). A saída da rede dita a ação (*Regar, Aplicar Defensivo, Andar para Frente, Virar*). Como não usaremos *Backpropagation*, os pesos e vieses (matrizes) dessa rede representam o "DNA" do agente.
2. **A Evolução (Algoritmo Genético):** O sistema inicializa uma população de 50 ou 100 robôs com pesos neurais aleatórios. Cada robô é solto em uma estufa simulada por um tempo limite. O Algoritmo Genético avalia o desempenho de cada um (sua função *Fitness*), seleciona os melhores "agricultores", realiza o cruzamento de suas matrizes de pesos e aplica pequenas mutações para gerar a próxima geração.

Requisitos de Implementação (Passo a Passo)

O código em Python deve ser estruturado nos seguintes módulos:

- **Passo 1: Motor da Simulação (A Estufa e o Robô)**
 - Codificar a classe do ambiente (uma matriz com plantas geradas aleatoriamente) e a classe do robô, garantindo que ele consiga ler o estado da célula onde está.
 - Definir a regra de pontuação (*Fitness*): +10 pontos por curar/regar uma planta corretamente, -2 pontos por gastar água à toa, -1 ponto por bater na parede.
- **Passo 2: Arquitetura da Rede Neural (Inferência)**
 - Utilizar numpy para construir o cálculo matemático do *forward pass*. O aluno não precisa usar bibliotecas pesadas de Deep Learning; basta multiplicar o vetor de entrada pela matriz de pesos, somar o viés e aplicar uma função de ativação simples (como ReLU ou Sigmoid).
- **Passo 3: O Algoritmo Genético (Otimização)**
 - Criar o loop evolutivo. Implementar a seleção (torneio ou roleta), o *crossover* (trocar partes dos vetores de pesos entre dois pais) e a mutação (adicionar um pequeno ruído aleatório em alguns pesos).
- **Passo 4: Monitoramento e Visualização**
 - Utilizar matplotlib para gerar um gráfico de linhas obrigatório mostrando o acompanhamento das gerações: uma linha para o *Fitness* máximo e outra para o *Fitness* médio da população. Isso é vital para provar que a evolução está ocorrendo.

Cr terios de Avalia  o

1. **Converg ncia do Modelo (40%):** O gr fico do matplotlib demonstra claramente uma curva de aprendizado ascendente? Os rob s das  ltimas gera  es se comportam visivelmente melhor que os da primeira?
2. **Modelagem da Fun  o de Fitness (30%):** A fun  o de avalia  o foi bem desenhada?
3. **Estrutura da Matriz de Pesos (30%):** O aluno conseguiu mapear corretamente os pesos do Algoritmo Gen tico (vetor 1D) para a estrutura da Rede Neural (Matrizes 2D) e vice-versa durante o *crossover*?

Projeto 3: Roteirizador Dinâmico (Sistemas Multiagentes + Algoritmos

,

O Cenário: A arquitetura coordena agentes autônomos de transporte que realizam entregas em uma malha logística (como pontos de distribuição regionais). O problema exige planejamento global estático combinado com execução local reativa.

A Abordagem Híbrida:

- **Módulo de Planejamento (Algoritmo Genético):** Antes dos agentes entrarem em movimento, o AG resolve o Problema do Caixeiro Viajante (TSP) para definir a ordem exata de visita dos pontos de entrega, minimizando a distância total.
- **Módulo de Execução (Sistemas Multiagentes):** Agentes reativos recebem a rota otimizada e iniciam a navegação na malha. Se um agente detecta um bloqueio repentino em uma via (simulado aleatoriamente no código), ele suspende a rota do AG e utiliza uma heurística simples para encontrar um desvio seguro até o próximo ponto válido.

Requisitos de Implementação:

- Modelar o ambiente logístico como um grafo ou matriz de adjacência.
- Criar o loop do AG para otimizar as rotas iniciais.
- Construir a classe do agente com regras de transição de estado claras (estado de "Navegação Normal" seguindo o AG vs. estado de "Recuperação de Rota").

Requisitos de avaliação:

1. A capacidade do agente de lidar com imprevistos dinâmicos no ambiente (vias bloqueadas) (30%). O sistema não pode falhar ou travar quando a rota inicial se torna inválida. O agente deve demonstrar uma lógica de *fallback* eficiente, sendo capaz de suspender temporariamente a rota calculada pelo Algoritmo Genético, identificar o bloqueio através de seus "sensores" simulados e acionar uma heurística de contingência para contornar o obstáculo com sucesso antes de retomar o plano original.

2. O desempenho do Algoritmo Genético ao resolver o Problema do Caixeiro Viajante (TSP) na fase de planejamento (30%). O código deve implementar corretamente os operadores evolutivos (seleção, cruzamento e mutação). Mais importante, a modelagem da função de *fitness* e a taxa de mutação devem evitar que a população fique presa precocemente em mínimos locais, garantindo que as rotas iniciais entregues aos agentes sejam caminhos globais altamente otimizados e lógicos.

3. A modularização do código e a clareza na separação das responsabilidades entre os dois paradigmas de Inteligência Artificial (40%). Deve haver uma fronteira clara no código: o Algoritmo Genético atua estritamente como um orquestrador prévio (nível macro), enquanto a classe do agente gerencia a execução local (nível micro). O aluno deve implementar uma máquina de estados bem definida para o agente, com transições claras entre o estado de "Navegação Normal" (seguindo o AG) e o estado de "Recuperação de Rota" (reativo).